Technical Report: C-Style Parser and AST Generation

Course Code: COD7001

Mayur Kamble 2025MCS2116

Nizaul Rahman 2025MCS2099

1. Introduction

The objective of this project was to design and implement a syntax analyzer (parser) for a subset of the C programming language. The parser converts a sequence of tokens provided by the lexical analyzer into an Abstract Syntax Tree (AST). This tree represents the hierarchical structure of the source code, facilitating future compilation stages such as semantic analysis and code generation.

2. Language Specifications

The supported language features include:

- **Variable Management:** Integer variable declaration and assignment.
- **Expressions:** Arithmetic expressions using `+`, `-`, `*`, `/`.
- **Boolean Logic:** Relational operators ==, `!=`, <, >, <=, >=.
- **Control Flow:** Nested `if-else` statements and `while` loops.
- **Scoping:** Statement blocks using curly braces {}.

2.1 Grammar Specification (BNF)

Rule Name	Production	Description
Program	statement_list	Entry point of the parser
Statement List	statement | statement_list statement	Sequence of statements
Statement	var_decl | assign | block	Executable unit
Block	{ statement_list }	Scoped block

If Statement	if (expr) stmt [else stmt]	Conditional execution
While Loop	while (expr) stmt	Iterative execution
Expression	INTEGER | ID	Atomic expression

2.2 Operator Precedence and Associativity

Precedence	Operator	Description	Associativity
1 (Highest)	( )	Parentheses / Grouping	N/A
2	$*, /$	Multiplication, Division	Left-to-Right
3	$+, -$	Addition, Subtraction	Left-to-Right
4	$<, >, <=, >=$	Relational Operators	Left-to-Right
5	$==, \neq$	Equality Operators	Left-to-Right
6 (Lowest)	$=$	Assignment	Right-to-Left

3. Design Methodology

3.1 Lexical Analysis (Flex)

The lexical analyzer identifies keywords, operators, identifiers, and constants. A global variable `line_num` tracks source line numbers for precise error reporting.

3.2 Syntax Analysis (Bison)

The grammar is written using Backus–Naur Form (BNF). Explicit operator precedence rules ensure deterministic parsing.

3.3 Dangling Else Resolution

The dangling `else` ambiguity is resolved using Bison's `%nonassoc` directives, ensuring correct association with the nearest unmatched `if`.

3.4 AST Data Structure

The AST uses a recursive node representation. A sibling pointer (`next`) allows sequential statements within blocks.

4. Implementation Details

- `ast.h / ast.c`: AST node definitions and traversal.
- `parser.y`: Grammar rules and semantic actions.

- `main.c`: Parsing control and AST visualization.

5. Testing and Validation

The compiler front-end was validated using 20 test cases.

5.1 Functional Verification (Valid Inputs)

Case	Category	Input Fragment	AST Verification
Test 1	Basic Arithmetic	`x + 5 * 2`	MULT nested under PLUS
Test 2	Scoping	`{ var result = 100; }`	NODE_BLOCK created
Test 3	Logic Nesting	`while(...) { if(...) }`	IF inside WHILE body
Test 4	Complex Logic	Mixed expressions	Full AST validated

Sample Trace of Test Case

```
--- Abstract Syntax Tree ---
VAR_DECL: alpha
  INT: 10
VAR_DECL: beta
  INT: 20
IF_STATEMENT
  CONDITION:
    BINOP: >
      BINOP: *
        BINOP: +
          ID: alpha
          ID: beta
        BINOP: -
          ID: gamma
          ID: alpha
      INT: 500
  THEN:
    BLOCK
      VAR_DECL: nested_val
      IF_STATEMENT
        CONDITION:
          BINOP: >=
            ID: nested_val
            INT: 1
```

```
    THEN:
      BLOCK
        ASSIGN: check
          INT: 1
```

5.2 Robustness and Error Handling (Invalid Inputs)

Case	Error Type	Sample Code	Reported Error
Inv 1	Expression Error	`(10 + 5) * / 2`	Syntax Error at line 3
Inv 2	Dangling Else	`else { ... }`	Syntax Error at line 5
Inv 3	Illegal Condition	`while (var i = 0)`	Syntax Error at line 4
Inv 4	Unclosed Block	`{ var x = 1;`	Syntax Error at line 10

6. Conclusion

The parser reliably converts token streams into a structured AST, correctly handling operator precedence, nested control flow, and syntax errors, providing a solid foundation for further compiler phases.